# Mini-Project Technical Report

## Abstract

**Project name:** Guru-Air Monitor

**Authors:** Jaakko Airikkala, Juhani Kataja, Eemil Sinkko

**Course:** Data Science (DATA11001)

**Document:** Technical report

**Date:** October 26th 2026

**Project repository:** https://github.com/juhanikat/air-data-project

**Abstract:**

The Guru-Air Monitor project collects data from air quality sensors installed in the Gurula student room at the Kumpula campus, and some other nearby rooms. This data is served through a Backend API, and visualized with our publicly available and easy to use Frontend. Finally, forecasts for the next two days can be generated for any sensor reading using a SARIMA statistical model. The project allows a user to, for example, view the time of day when the temperature of the Gurula fridge will likely be at its lowest, and plan the restocking of the fridge around that time. Alternatively a user could track and predict room usage patterns based on CO2 measurements.

**Demo URL:** http://icetea.esinko.net:9000

**Demo credentials:** Username: *review*, password: *HYr3v1ew!*

**Demo instructions:**

Use a computer and a modern web browser. Navigate to the demonstration URL. Navigate the webpage via buttons in the page header section. Login by clicking *"Login"* on the top right to access all features. Click *"Measurements"* to view the latest measurements. Click *"Statistics"* to view hourly mean calculations for the past 2 weeks. Click *"Forecast"* to perform forecasts with select parameters (sensor, term). Note that forecasting can take up to 30 seconds! In the forecasts daily minimums are highlighted green and maximums highlighted red. Click on *"Settings"* to configure sensor labels and location.

# Technical architecture and design

The technical architecture of our course project was designed around our data collection and presentation method of choice. These choices formed a system we called the *Data Pipeline* as data flows through every component before being presented to the user. This architecture is showcased in *figure 1*. The pipeline consists of 5 parts: data collection (1), transmission (2), pre-processing (3), post-processing (4) and presentation (5). This approach was inspired by the OSI Model [1].

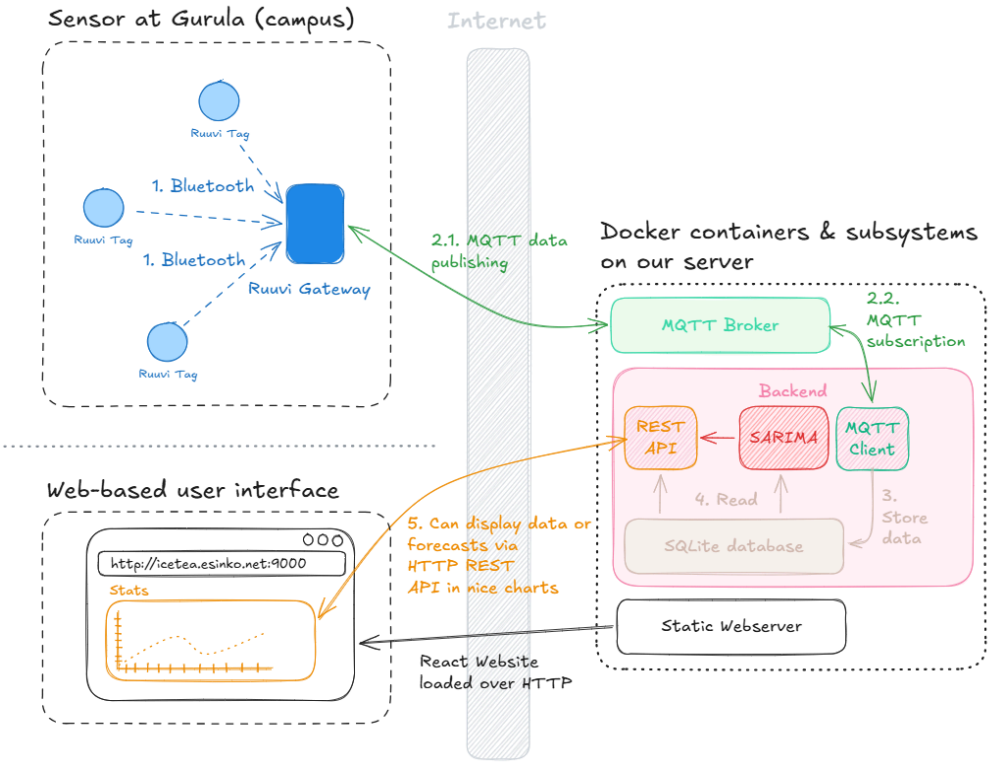



*Figure 1: Architecture diagram in which flow of data is indicated with arrows*

## Data Collection (1) and privacy considerations

All measurements, such as air temperature, humidity and the presence of certain pollutants, are being collected by six individual Ruuvi brand sensors. Five of these sensors are Ruuvi Air sensors, which provide the most data [2]. They are placed at around head height in different rooms. The 6th sensor is a Ruuvi Tag Pro sensor placed inside the Gurula students cafeteria (RV) drinks fridge [3].

Measurements from all the sensors are broadcast over Bluetooth[4] or Bluetooth Low Energy[5] to anyone within listening distance. However, we chose to specifically avoid collecting raw data due to privacy risks. Bluetooth broadcast data contains a lot of information regarding other devices present in the spaces too. We feared we might unintentionally create a database that could be used to track people's activities in the rooms. Therefore, we decided to use the Ruuvi Gateway, which listens to Bluetooth broadcasts for us and only sends sensor measurements forward [6]. This doesn't entirely remove the privacy risks, but places the responsibility on Ruuvi instead.

## Data transmission (2)

The Ruuvi Gateway supports two protocols for data transmission: HTTP(S) and MQTT. We used the MQTT (Message Queuing Telemetry Transport) protocol as HTTP(S) options on the Gateway were already in use by other systems. This introduced a lot of complexity in our architecture, since MQTT operates via a Publish/Subscribe model instead of the traditional Client/Server model (like HTTP). This means we needed an MQTT Broker to queue up the messages from the Gateway and a separate MQTT Subscriber in our own software to read from these message queues [7].

Since we had to incorporate multiple individual software components in our architecture, we used Docker and Docker Compose to containerize them [8]. These products allow us to define our architecture in text, which allows us to easily deploy development and production environments despite our architectural complexity. For the MQTT Broker, we chose to use an existing open source Broker software called Mosquitto by the Eclipse foundation as it had an existing Docker container image available [9].

## Data processing (3, 4) and the Backend

Data pre- and post-processing functionalities are implemented in the core part of our infrastructure called the Backend, which is the 2nd Container. It implements an MQTT Subscriber to collect measurement data, custom SQLite3 database abstractions to store it and a REST API[10], which provides access to data, authenticates users and performs on-demand data downscaling[11] and forecasting. Additionally the Backend features a terminal interface for user management purposes.

The Backend is implemented in Python 3.13 utilizing various libraries, such as Paho-MQTT[12] for the MQTT subscriber, Flask[13] for the REST API and a combination of Statsmodels[14], Pandas[15] and Numpy[16] for data processing. Poetry was used for dependency management [17]. The Backend represents a clear majority of work done for this project. Implementing the core features and optimizing them took a lot of time and many iterations as our goals for the project shifted.

## Data presentation (5) and the Frontend

We developed a web page for interactive data presentation. The page was implemented in React [18] using Typescript [19] with the Vite framework [20]. We also utilized a components library called Chakra UI [21], recharts [22] for drawing time-series charts and some icons from react-icons [23]. NPM was used for dependency management on the Frontend and an exact dependency list is available in the project repository (web/package.json)[24].

Frontend design started by workshopping ideas and drawing crude skeletons of the layout. Styling was perfected incrementally and responsivity was incorporated after everything looked good. In the end we settled on a rather traditional header-based navigation bar and a minimalistic design. In the production environment, we used nginx (3rd Container) to serve the web page [25].

# Machine Learning

We used the SARIMA (seasonal autoregressive integrated moving average) model to create the predictions [26]. We chose SARIMA because it can be used to predict time series data that has predictable seasons, and because of its relative simplicity and easy implementation using Python.

## Initial predictions and planning

*Figure 2* shows the temperature data for sensor 1 over roughly five weeks. We expected to see some seasonality within the days and the data seems to confirm this. There could also be seasonality within weeks and years, but our limited data cannot confirm this. There appears to be an increasing trend in our data, which we did not expect, because the values we measure should stay around some mean. Fluctuations like this could make the SARIMA model less accurate, but more data would be needed to know how well the model fits the data and what parameters we should use.

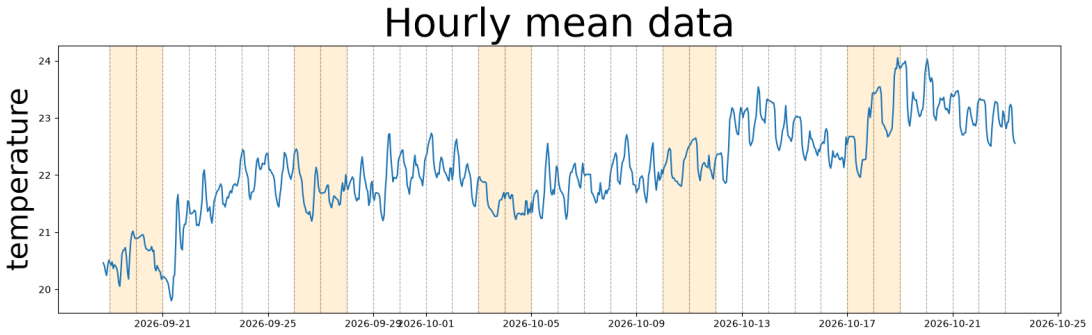



*Figure 2: Hourly mean temperature data from 18.9.-23.10.2026. Shaded areas are weekends*

## Model training

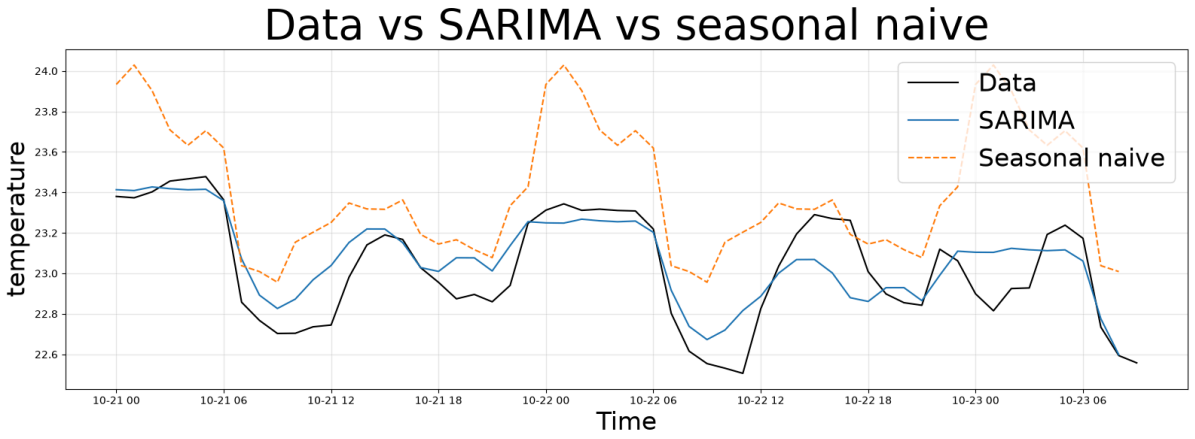



*Figure 3. Predictions compared to the testing data. Predictions span 57 hours*

We split the data into a training set and a testing set (last 57 hours of data), and fitted the SARIMA model on the training set. Because of limited time, we chose to tune the parameters of our model only for the temperature readings of sensor 1. The predictions may not generalize as well to other sensors and data types. *Figure 3* shows both the SARIMA predictions (blue line) and real measurements (black line) for the sensor 1. As a baseline, we

compared the model to a seasonal naive forecast, which simply uses the values from the same time the previous day (dashed orange line in *figure 3*). SARIMA follows the data more closely. However, this comparison covers only one 57-hour period, so the result may be different for other time frames.

# Reflection

## Outcome and changes from the initial plan

Initially, our goal was to gather data from the temperature sensor inside the drinks fridge in Gurula, and use this data to find an optimal time to fill the fridge. We succeeded at this goal, but over time we also decided to focus on the air quality readings (such as CO2) from the other sensors, and use them to estimate the number of people using the rooms around Gurula. In the end, our finished product has expanded functionality compared to the one described in the mini-project canvas, even though we weren't able to produce any specific predictions for the number of people present in the rooms.

## Experience gained, what we would have done differently and possible future changes

In summary, we got to deep-dive into practical data science. It was fun, but challenging! In the end more work was needed than expected, but we are happy with our results. Clear highlights for us were: learning to use new data-science and frontend libraries and learning a bunch about time-series forecasting. In the future, with more data, it would be interesting to tune the model parameters further and test how well the model performs. We could also fit separate parameters for each sensor and data type to get more meaningful results.

Juhani: I focused more on frontend development during the project, and learned a lot about the Chakra UI component library and about data visualization in general. As with any software project, sometimes more work was needed to reach a deadline than I initially thought, and new libraries proved more difficult to learn than anticipated. Working on the project more regularly would have made development easier and maybe even resulted in a better looking frontend.

Jaakko: My focus was on understanding how to use the SARIMA model and how to calculate predictions using Python libraries. I learned a lot about data handling and preprocessing with Pandas. SARIMA was completely new to me and it was fun searching tutorials and reading about it online.

Eemil: My focus was designing and implementing the core architecture. I got to do a lot of coding. A lot of old stuff I already knew, but also new stuff. Everything related to data processing with Pandas, Numpy or Statsmodels was new stuff. Optimizing their usage in our implementations took more tinkering than I thought and I really got to challenge myself. In the future we should optimize them further and create forecasts with a specific schedule and not on-demand. That stuff is CPU intensive!

# References


[1] OSI Model, Wikipedia.org (https://en.wikipedia.org/wiki/OSI_model, visited 26.10.2026)

[2] Ruuvi Air sensor, Ruuvi.com (https://ruuvi.com/i/u/Ruuvi-Air.pdf, visited 26.10.2026)

[3] Ruuvi Tag Pro sensor, Ruuvi.com (https://ruuvi.com/i/u/ruuvitag-tech-spec-2023-04.pdf, visited 26.10.2026)

[4] Bluetooth, Wikipedia.org (https://fi.wikipedia.org/wiki/Bluetooth, visited 26.10.2026)

[5] Bluetooth Low Energy, Wikipedia.org (https://en.wikipedia.org/wiki/Bluetooth_Low_Energy, visited 26.10.2026)

[6] Ruuvi Gateway, Ruuvi.com (https://ruuvi.com/fi/gateway/, visited 26.10.2026)

[7] MQTT, Wikipedia (https://en.wikipedia.org/wiki/MQTT, visited 26.10.2026)

[8] Docker (software), Wikipedia.org (https://en.wikipedia.org/wiki/Docker_(software), visited 26.10.2026)

[9] Mosquitto by the Eclipse foundation, mosquitto.org (https://mosquitto.org/, visited 26.10.2026)

[10] REST, Wikipedia.org (https://en.wikipedia.org/wiki/REST, visited 26.10.2026)

[11] (Data) Downscaling, Wikipedia.org (https://en.wikipedia.org/wiki/Downscaling, visited 26.10.2026)

[12] Pahp-MQTT, Pyp.org (https://pypi.org/project/paho-mqtt/, visited 26.10.2026)

[13] Flask, Pypi.org (https://pypi.org/project/flask/, visited 26.10.2026)

[14] Statsmodels, Pypi.org (https://pypi.org/project/statsmodels/, visited 26.10.2026)

[15] Pandas, Pypi.org (https://pypi.org/project/pandas, visited 26.10.2026)

[16] Numpy, Pypi.org (https://pypi.org/project/numpy, visited 26.10.2026)

[17] Poetry, python-poetry.org (https://python-poetry.org/docs/, visited 26.10.2026)

[18] React (software), Wikipedia.org (https://en.wikipedia.org/wiki/React_(software), visited 26.10.2026)

[19] TypeScript, Wikipedia.org (https://fi.wikipedia.org/wiki/TypeScript, visited 26.10.2026)

[20] Vite, Wikipedia.org (https://en.wikipedia.org/wiki/Vite, visited 26.10.2026)

[21] Chakra UI, npmjs.com (https://www.npmjs.com/package/@chakra-ui/react, visited 26.10.2026)

[22] Recharts, npmjs.com (https://www.npmjs.com/package/recharts, visited 26.10.2026)

[23] React Icons, npmjs.com (https://www.npmjs.com/package/react-icons, visited 26.10.2026)

[24] npm, Wikipedia.org (https://en.wikipedia.org/wiki/Npm, visited 26.10.2026)

[25] nginx, nginx.org (https://nginx.org/, visited 26.10.2026)

[26] Autoregressive integrated moving average, Wikipedia.org (https://en.wikipedia.org/wiki/Autoregressive_integrated_moving_average, visited 26.10.2026)